

The FT1 Protocol — How It Works & What Makes It Different

A technical companion to the Tempo product brief.

Tempo is an off-grid, weak-signal **text** application for amateur radio, by Seth McCallister, KD9TAW.

Repository: github.com/kd9taw/tempo · Manual: see [tempo/docs/manual/](#) in the repo · License: GPL-3.0.

Honesty banner, up front. Every sensitivity number in this document is **simulation-validated only** — AWGN and Rayleigh-fading sweeps in the project's test harnesses ([ft1_test_standalone](#) , [dx1_test](#) , re-validated in the Rust test suite) — and is **not** on-air proven. Where the implementation does not yet expose a feature, that is stated plainly. On-air decode-rate-vs-SNR validation is the project's #1 remaining gate. Read accordingly.

1. What makes FT1 different (the short version)

If you already know FT8, FT4, and JS8, here is the entire pitch in three bullets. The rest of this document unpacks each one.

- **It is coherent.** FT1 uses **4-CPM** — 4-ary *continuous-phase modulation* — and is designed to demodulate it **coherently** (tracking carrier phase), where FT8/FT4/JS8 demodulate Gaussian-smoothed FSK **non-coherently** (energy per tone, no phase tracking). Coherent detection extracts more information per second of air time. That is the lever FT1 pulls to make a fast cycle competitive.
- **It has IR-HARQ.** FT1's design carries an **incremental-redundancy hybrid-ARQ** scheme: a retransmission sends *new parity bits the receiver has never seen*, and the receiver soft-combines them with the original frame to decode a longer, lower-rate code. No FT8/FT4/JS8 mode does this — they are one-shot block codes per frame. This is the headline differentiator.
- **It is conversational.** FT1 runs a **4-second** T/R (transmit/receive) cycle — versus 15 s for FT8, 7.5 s for FT4, and 15/10/6/30 s for JS8. Short cycles make a back-and-forth text exchange feel like a conversation instead of a slideshow.

The honest counterweight, stated here so it frames everything below: **FT1 trades roughly 3 dB of raw single-shot sensitivity for that speed.** Its simulated AWGN 50%-decode threshold is about **−15 dB** in a 2500 Hz reference bandwidth; FT8 is commonly cited at about **−21 dB**. **FT1 does not beat FT8 on raw sensitivity.** What it offers is *speed*, plus an *IR-HARQ path* (designed and simulated, not yet live — see §6 and §9) that claws sensitivity back over multiple short transmissions instead of one long one.

Jargon, defined once. T/R cycle — the fixed clock that alternates transmit and receive, aligned to UTC. **SNR in 2500 Hz** — signal-to-noise ratio referenced to a 2500 Hz noise bandwidth, the standard yardstick for these modes, so the numbers are comparable. **Coherent vs non-coherent** — whether the receiver tracks the carrier's *phase* (coherent: more sensitive, fragile to fading) or only its *energy* (non-coherent: more robust, less sensitive). **LDPC / FEC** — forward error correction; parity added so the receiver fixes bit errors without a resend. **LLR** — log-likelihood ratio; a soft, signed confidence value per bit (sign = guess, magnitude = reliability).

2. The design goal and the physics it is fighting

Weak-signal text faces a tradeoff that is **fundamental physics, not an engineering shortfall**: *cycle time versus weak-signal reach*.

You can integrate the signal over a long window to dig it out of the noise, or you can keep the cycle short so the conversation flows — but **every second you remove from the integration window costs sensitivity**. There is no single waveform optimal at both ends of that curve.

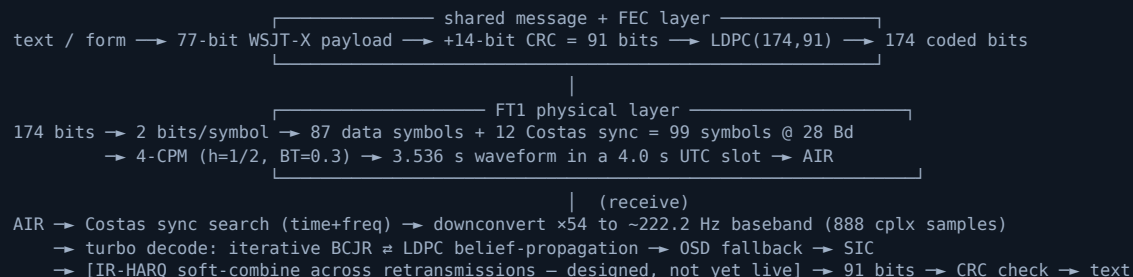
This is why Tempo ships **two tiers** rather than one compromise waveform:

- **FT1 (Fast tier)** — a 4 s cycle, coherent, the most information per second of air time, but it leans on carrier-phase stability that fading and Doppler spreading destroy.
- **DX1 (Robust tier)** — a 15 s cycle, non-coherent, slower but survives the fading that collapses coherent modes.

Both tiers carry the **identical 77-bit message payload** and the **same LDPC(174,91) FEC** (see §4). The tier is never silent: it is a deliberate, operator-driven toggle surfaced in the UI. The engine reads that choice to pick which waveform to modulate/decode and which slot clock to run. **The abstraction hides the DSP, not the decision.**

2.1 The signal chain at a glance

Both tiers share the message and FEC layers; they diverge at modulation, sync, and the slot clock. The FT1 path:



3. The FT1 waveform

FT1 is, in one line: *4-CPM, $h=1/2$, $BT=0.3$, 99 channel symbols at 28 Bd, designed for coherent demodulation with iterative turbo equalization.*

3.1 Modulation: 4-ary continuous-phase modulation (4-CPM)

- **4-ary** — each symbol carries 2 bits (4 states).
- **Continuous-phase** — the carrier phase never jumps; it advances smoothly across symbol boundaries. Continuity keeps the spectrum tight and makes the signal amenable to coherent, trellis-based detection.
- **Modulation index $h = 1/2$** — controls how far the phase advances per symbol. $h = 1/2$ is the classic minimum-shift-style index that balances spectral compactness against the distance between symbols (which sets sensitivity).
- **Gaussian $BT = 0.3$** — the frequency pulse is shaped by a Gaussian filter with time-bandwidth product 0.3. Lower BT means a gentler, more bandwidth-efficient pulse, at the cost of more inter-symbol memory — which the trellis detector is built to exploit rather than fight.

Jargon. CPM (continuous-phase modulation): a phase-continuous waveform family; the receiver can treat the phase trajectory as a trellis and detect it with maximum-likelihood / BCJR methods. **Modulation index h :** the per-symbol phase advance, in units of π . **BT (time-bandwidth product):** how tightly the Gaussian pulse is shaped — smaller is narrower-band.

3.2 Symbol and frame structure

Quantity	Value	Note
Total channel symbols	99 (FT1_NN = 99)	12 sync + 87 data
Sync symbols	12	three 4×4 Costas arrays
Data symbols	87	174 coded bits ÷ 2 bits/symbol
Sample rate	12 kHz	standard WSJT-X audio rate
Symbol (baud) rate	28 Bd	12000 ÷ (3000/7); 428.57 samples/symbol nominal (429 rounded)
Waveform duration	≈ 3.536 s	inside the frame
T/R period	4.0 s	UTC-aligned (PERIOD_S = 4.0)
Pre-TX guard	200 ms	silence before the waveform (PRE_TX_GUARD_MS = 200)
Timing tolerance	±80 ms	slot-boundary alignment (TIMING_TOLERANCE_MS = 80)

The 3.536 s waveform inside a 4.0 s frame leaves roughly **0.464 s** for receive guard plus decoding. The three Costas sync arrays are interleaved with the data (beginning / middle / end style placement), which is what makes the time/frequency search in §5 tractable.

3.3 Occupied bandwidth

FT1's occupied bandwidth is **not published as a constant in source** — there is no authoritative figure in `ft1_params.f90` or the Rust modem. Calculated estimates **range roughly 42-67 Hz depending on method**. Treat FT1's occupied bandwidth as *designed to be narrow* and quote it as "**not specified (est. ~42-67 Hz)**" rather than as a hard spec. (Older design discussion floated larger round numbers; none is grounded in source and none should be cited.)

3.4 Coherent demodulation + turbo equalization

This is where the "most information per second" claim comes from. FT1's receive chain is iterative:

1. **Costas sync candidate search** over time and frequency offsets (see §5).
2. **Downconvert** the candidate to complex baseband. The modem **downsamples by 54** (`FT1_NDOWN = 54`), taking the audio frame to **~222.2 Hz** ($12000 \div 54$) and yielding **888 complex baseband samples** (`FT1_NDMAX = 888`) for the matched-filter bank. That is about **9 downsampled samples per channel symbol** ($888 \div 99$); the source comment rounds this to "**~8 samples/symbol**."
3. **Turbo decode** (`ntype = 1`) — iterative **BCJR** (a maximum-likelihood trellis detector for the CPM phase memory) exchanges soft information back and forth with **LDPC belief-propagation** decoding. Each pass refines the other.
4. **OSD fallback** (`ntype = 2`) — ordered-statistics decoding when belief propagation alone does not converge.
5. **SIC** — successive interference cancellation (signal subtraction) to peel a decoded signal off and expose weaker ones.
6. **IR-HARQ combining** — soft-combine across retransmissions (see §6; designed, not yet exposed in the live decode path).

The iterative loop (step 3) is where FT1's headroom over a vanilla one-shot LDPC decode is expected to come from. The project's **design estimate is ~1.5-2 dB** of gain from iterative turbo equalization — i.e., a baseline near **-15 dB with iterative detection** vs roughly **-14 dB without**. This is a **medium-confidence design estimate**, not a measured figure.

Implementation note. The FT1 modem uses process-global `SAVE` state (CPM pulse tables, the downsample window, cached FFTW plans) and is **not thread-safe**, so every entry point serializes behind a global `MODEM_LOCK: Mutex<()>`. The real-time path (`decode_rt`) assumes a frame-aligned buffer (`dt0 = 0` , ± 3 downsampled-sample search); full over-the-air acquisition (Costas sync + frequency/time search) is a planned `libft1` ABI addition.

4. The message layer

FT1 deliberately reuses the **WSJT-X 77-bit message format**, which is the same source coding FT8, FT4, and JS8 all use. This is not an accident — it is the whole interoperability story.

- **77-bit payload.** Structured messages (callsigns, grids, reports) and free text pack into 77 bits exactly as in WSJT-X. Because of this, **Chat, QSO, and Field-Day forms ride identically** on FT1 — and on DX1, and (eventually) on the FT8 tier.
- **14-bit CRC.** Appended to the 77 message bits → **91 bits total** (`KK = 91`).

- **LDPC(174,91) FEC.** The 91 bits encode to **174 coded bits** (83 parity bits added). At 2 bits per channel symbol, that is the 87 data symbols in §3.2.
- The decoded result is the 91 bits back out (77 message + 14 CRC); the CRC confirms the codeword before the text is unpacked.

The 77-bit packer is reused directly from WSJT-X (GPLv3 heritage), so a fragment that fits the standard message structure is bit-for-bit a WSJT-X-style payload.

4.1 Chunking longer free text

A single 77-bit payload holds only a short fragment of free text. Longer messages are **chunked across multiple 4 s frames** at the application layer — each frame carries one valid 77-bit payload, and Tempo reassembles the sequence into the chat stream. The fast 4 s cycle is what makes multi-frame free-text feel responsive rather than glacial.

5. Synchronization and acquisition

FT1 syncs on **three 4×4 Costas arrays** (12 sync symbols total). A *Costas array* is a frequency-hop pattern with a sharp, near-ideal autocorrelation — it lights up exactly once when time and frequency are aligned, which is what makes a 2-D search practical.

Jargon. Costas array: an N×N frequency-hop pattern with a single sharp autocorrelation peak, used as a sync marker that survives noise and frequency offset.

- **Full acquisition** correlates the Costas pattern across a grid of **time offsets and frequency offsets** (a frequency band from `nfa` to `nfb` Hz). The candidates that correlate strongly become decode attempts.
- **Real-time decode** (`decode_rt`) assumes the frame is already aligned (`dt0 = 0`) and searches only **±3 downsampled samples** of fine timing. This is the loopback / known-timing path; it is sufficient for testing and frame-aligned reception but not for arbitrary over-the-air timing.

Beyond the **±3 downsampled-sample** real-time tolerance, the exact frequency/time search tolerances are not specified here, and the A-priori (AP) decoding schedule is not detailed — treat these as implementation internals, not published specs.

5.1 The dt convention

FT1 reports time offset using the WSJT-X convention: **$dt = t - 0.5$** , where t is arrival time into the frame. So **$dt \approx -0.1$ s means the signal arrived at 0.4 s** into the cycle. This centers the nominal frame boundary at $dt = 0$ and matches what WSJT-X operators already expect.

6. The differentiator: incremental-redundancy HARQ (IR-HARQ)

This is the part that has no analog in FT8, FT4, or JS8.

Read this first. The redundancy-version (RV) mechanism is a **documented protocol design**. The IR-HARQ combine module is reset (`harq_init()`) at the start of each frame to keep frames independent, and an `rv` field exists in the result struct — **but the decode callback does not expose it (it reports `rv = -1`), and soft combining is not yet wired end-to-end** in the live FT1 decode path. Everything in this section is therefore **designed and simulated, not shipping, on-air-proven behavior**. The RV0/RV1/RV2 progression below is the documented design, not an observable runtime contract.

6.1 The problem with just resending

When a weak-signal frame fails to decode, the obvious fix is to resend it. Two ways exist:

- **Chase combining** — resend the *same* 174 coded bits. The receiver averages the two copies, gaining ~3 dB of energy per retransmission. Simple, but you are paying full air time to re-hear bits you already have.
- **IR-HARQ (incremental redundancy)** — resend *new parity bits the receiver has never seen*. The receiver combines the original 174-bit LLR vector with the fresh parity LLRs, **effectively decoding a longer, lower-rate code**. You get *both* the energy accumulation of chase combining *and* extra coding gain from the lower effective code rate.

IR-HARQ is the strictly-superior option, and it is FT1's signature.

6.2 How the redundancy accumulates (documented design)

FT1's IR-HARQ design is **rate-compatible**, built from a mother code extended above the baseline:

Transmission	Bits sent this TX	Effective code	Cumulative
RV0 (1st TX)	174	LDPC(174,91)	baseline
RV1 (2nd TX)	87 <i>new</i> parity	LDPC(261,91)	original + new parity
RV2 (3rd TX)	87 <i>new</i> parity	LDPC(348,91)	mother code

The mother code is **LDPC(348,91)**, extended from the baseline LDPC(174,91). The 174 → 261 → 348 progression is sourced from the protocol design document; the exact schedule and the number of RVs defined are **not observable from the decode callback** (`rv = -1`), so read this table as the documented design.

Jargon. RV (redundancy version): which slice of the mother code a transmission carries. **Puncturing:** sending only a subset of a longer code's bits; the puncture pattern is optimized to maximize first-TX (RV0) decode probability while keeping the incremental gain high.

6.3 Why the combining is principled

The receiver combines by **adding LLRs**: `LLR_combined = LLR_1 + LLR_2 + ...`. For independent AWGN between transmissions this is **maximum-likelihood optimal** — the magnitudes already encode reliability, so no explicit SNR weighting is needed. On fading channels, the ~4 s gap between transmissions means each TX likely sees a *different* fade state, so the combine also buys **time diversity** for free.

Why IR beats chase at *real* decoder operating points (not just at the Shannon limit, where 2-TX IR and 2-TX chase are near-equivalent at ~-18.5 dB): practical belief-propagation decoders converge more reliably on lower-rate codes, the finite-length penalty shrinks at lower rates, and *new* parity bits carry more information than repeated identical bits. Net: **~1-2 dB at practical operating points**.

6.4 Zero signaling overhead

The redundancy version is carried by **Costas-array pattern variants** (RV0 uses the standard pattern; RV1/RV2 use modified patterns). The receiver identifies the RV *during sync acquisition*, before data demodulation, so it knows which slice to expect. **Total extra bits for IR-HARQ: zero.** It rides entirely on existing protocol structure.

6.5 What this buys, in simulation

All figures here come from the IR-HARQ protocol design analysis and assume idealizations (independent AWGN, no QRM, no operator delay). They are **simulation/design figures, not guarantees and not on-air results.**

- IR-HARQ's sweet spot is the **−17 to −20 dB** SNR range. At −20 dB, simulated throughput is ~4.5 bps for IR vs ~0.3 bps for chase combining.
- **3-TX IR-HARQ reaches roughly a −21 to −21.8 dB threshold** (design analysis) — *on par with, and possibly a touch beyond, FT8's commonly-cited −21 dB* — in **12 seconds total** (3 × 4 s).
- A full QSO at −20 dB ("typical DX conditions") completes in **~43 s** with IR-HARQ — still faster than FT8's ~60 s at the same SNR, despite needing multiple retransmissions per message.
- **Backward compatible:** a non-IR station just decodes RV0 (standard LDPC(174,91)), unaware of the RV mechanism. If an IR station's RV1 is not acknowledged, it falls back to resending RV0 so a legacy station can chase-combine two standard copies.
- **Cheap to track:** at most **348 bytes** of 8-bit LLRs per in-progress station (174 + 87 + 87). 100 simultaneous in-progress decodes ≈ 34 KB. LLRs are discarded after ~30 s (7–8 T/R periods) with no matching transmission. The per-frame `harq_init()` reset keeps each frame's combine independent of stale state.

7. The two tiers: FT1 (Fast) vs DX1 (Robust)

Both tiers carry the **same 77-bit payload and the same LDPC(174,91) FEC**. Only the modem, frame length, and T/R clock differ.

7.1 FT1 — Fast, coherent

- 4-CPM, coherent turbo equalization, 4 s cycle, ~−15 dB AWGN threshold (simulated).
- **Wins** when the path is stable: short-to-medium haul, ground wave, quiet bands, and any time you want a real conversation cadence.
- **Losses** under fading/multipath. Coherence is the Achilles heel: *because it is coherent it extracts the most information per second of air time — but coherence is exactly what multipath/Doppler spreading destroys.*

7.2 DX1 — Robust, non-coherent

DX1 (the DX1-S baseline) is the answer to FT1's fragility on bad paths.

- **Modulation:** non-coherent **8-FSK** — M = 8 orthogonal tones, 3 bits/symbol, **Gray coded**.
- **Rate / spacing:** baud = 6.25 Hz, tone spacing = baud = 6.25 Hz → **occupied data BW = 8 × 6.25 = 50 Hz.**

- **Sampling:** 12 kHz, **NSPS = 1920 samples/symbol**; a 1920-point FFT gives exactly 6.25 Hz/bin, so each of the 8 tones lands cleanly on its own bin.
- **Frame:** linear-**chirp preamble** (~0.64 s, 4 symbol periods, swept across the 50 Hz band for time/frequency sync) + **58 data symbols** ($174 \div 3$). On-air frame = **119,040 samples (9.92 s)** inside a **15 s** T/R slot (capture window 184,320 samples $\approx$ 15.36 s).
- **Decode:** chirp sync $\rightarrow$ per-symbol FFT energy per bin $\rightarrow$ **soft-decision LDPC** belief propagation. No carrier phase tracked anywhere.

Why it exists: *because it never relies on carrier phase, DX1 survives fading that collapses coherent modes.* In simulation, DX1's AWGN 50% threshold is $\approx$ **-18.6 dB** and it loses only **~3.7 dB** under per-symbol Rayleigh fading — where coherent FT8-class modes lose 10+ dB. **That small fading penalty is the entire reason the mode exists.**

DX1 known limit: the DX1 receiver is **narrowband** — it decodes at the operator's **tuned RX offset** (click the waterfall to set the carrier). Full-band DX1 search (decoding every signal across the passband in one pass) is **not** implemented; it is a Phase-2 roadmap item.

7.3 When each wins

- Stable path, want conversation speed $\rightarrow$ **FT1**.
- Fading/multipath/marginal DX, willing to slow down for reliability $\rightarrow$ **DX1**.
- Both reach roughly the same *messages*, just at different speed/robustness tradeoffs — and the operator picks, every transmission, from a visible toggle.

8. Fair comparison to FT8 / FT4 / JS8

The honest framing: **FT1 trades raw sensitivity for speed and adds an IR-HARQ path.** It is *not* a sensitivity win over FT8.

	FT1 (Fast)	DX1 (Robust)	FT8	FT4	JS8 (Normal)
Modulation	4-CPM (h=1/2, BT=0.3)	8-FSK	8-GFSK	4-GFSK †	8-FSK (JS8) ‡
Demodulation	Coherent (design)	Non-coherent	Non-coherent	Non-coherent	Non-coherent
Channel symbols	99 (12 sync + 87 data)	4 sync + 58 data	79	105	FT8 frame
Symbol rate	28 Bd	6.25 Bd	6.25 Bd	~23.4 Bd	6.25 Bd
Occupied BW	not spec. (~42-67 Hz est.)	50 Hz	~50 Hz	~90 Hz	~50 Hz
T/R cycle	4 s	15 s	15 s	7.5 s	15 s (also 30/10/6)
TX time / cycle	~3.54 s	~9.92 s	~12.6 / 12.64 s	~4.48 s	~12.6 s
Payload	77-bit	77-bit	77-bit	77-bit	77-bit
FEC	LDPC(174,91) + CRC-14	LDPC(174,91) + CRC-14	LDPC(174,91) + CRC-14	LDPC(174,91) + CRC-14	LDPC(174,91) + CRC-14
Sync	3 × 4×4 Costas	linear chirp	3 × 7×7 Costas	Costas	7×7 Costas
HARQ / incr. redundancy	Yes (IR-HARQ, designed)	No	No	No	No
AWGN threshold (2500 Hz)	~-15 dB (<i>sim</i>)	~-18.6 dB (<i>sim</i>)	~-21 dB	~-17.5 dB	(FT8-class)
Fading penalty	large (coherent)	~3.7 dB (<i>sim</i>)	10+ dB (coherent-class)	—	—

† FT4 is described as **4-GFSK** (four-tone FSK with Gaussian smoothing per the K1JT/K9AN/G4WJS protocol paper). Some sources use the looser label "4-MFSK"; because the protocol applies Gaussian smoothing, **4-GFSK is the accurate description**. ‡ JS8 = "**J**ordan **S**herer designed **8**-FSK," built directly on FT8's 8-FSK frame; characterize it only as far as the sourced statement allows.

Key reads from the table:

- **Sensitivity.** FT8 (~-21 dB) is the most sensitive single-shot mode; FT1 (~-15 dB) sits roughly where FT4 does (~-17.5 dB), give or take. FT1 gives up ~3 dB to FT8 in exchange for a *much* shorter cycle and the IR-HARQ path. This is directly analogous to how **FT4 itself trades ~3.5 dB of sensitivity vs FT8 for roughly half the cycle time** — FT1 just pushes that lever further and adds incremental redundancy. (*Note: some JS8 secondary sources quote FT8 theory near -24 dB; that is not the standard operational figure — use ~-21 dB.*)
- **The structural difference.** Every one of FT8/FT4/JS8 is a **one-shot LDPC block code per frame** with **no HARQ**. Reliability comes from strong FEC plus the operator manually repeating whole transmissions. FT1's IR-HARQ is the only scheme here that *accumulates redundancy* across retransmissions — and it is FT1's own design, not benchmarked head-to-head against these modes on the air.
- **The other structural difference.** FT8/FT4/JS8 are **non-coherent** (no carrier-phase tracking — a fundamental property of incoherent FSK detection). FT1 is **coherent CPM** by design, which is what lets it be fast *and* useful — at the cost of fading fragility, which DX1 exists to cover.

- **JS8 context.** JS8Call is built directly on the FT8 frame (same LDPC(174,91)+CRC, 7×7 Costas) plus a directed-calling / keyboard-to-keyboard / heartbeat / relay / store-and-forward layer. It is the closest existing *conversational* analog to what Tempo does — but it inherits FT8's non-coherent, no-HARQ transport and its 15 s baseline cycle (with selectable 30/10/6 s modes).

Comparison sourcing note. The FT8/FT4/JS8 figures here are web-sourced. FT1/DX1 numbers are the project's own **simulation** figures and were *not* benchmarked against these modes on the air. Do not read FT8/FT4/JS8 thresholds beyond the cited ~ -21 dB / ~ -17.5 dB values.

9. Status and caveats (read before you draw conclusions)

This is a beta, and the project says so plainly. The relevant honest caveats:

- **Simulation-only sensitivity.** Both FT1 (~ -15 dB AWGN) and DX1 (~ -18.6 dB AWGN, ~ 3.7 dB fading penalty) thresholds are **bench numbers** from AWGN and Rayleigh-fading sweeps (`ft1_test_standalone` / `dx1_test` , re-validated in the Rust suite). They are **not on-air proven**. Real propagation, QRM, and operator behavior are not in the simulation.
- **On-air validation is the #1 gate.** Decode-rate-vs-SNR on real bands is the top Phase-2 roadmap item and the thing that blocks calling Tempo operationally reliable. Honest on-air reports (band, dial, tier, distance, conditions, decodes vs. expectations) are the single most useful contribution.
- **IR-HARQ is designed, not live.** The protocol carries the RV field and the combine module resets per frame, but the decode callback does not expose RV (it reports `rv = -1`) and soft combining is not yet wired end-to-end — it is a Phase-2 item. The throughput/threshold gains in §6 are *designed and simulated*, not demonstrated.
- **FT8/FT4 tier is Phase 2.** The `Tier::Ft8` variant exists and the FT8/FT4 DSP sources compile into `libft1` , but **no end-to-end FT8/FT4 decode pipeline is wired**. Do not treat that tier as functional yet.
- **DX1 is narrowband.** It decodes only at the tuned RX offset; full-band search is Phase 2.
- **Low-confidence details are flagged as such.** The ~ 1.5 – 2 dB turbo-equalization gain is a *medium-confidence design estimate*. FT1's exact occupied bandwidth is *not* a published constant (est. ~ 42 – 67 Hz). Where this document says "designed to" rather than "is," that wording is deliberate.
- **Coherence is a design claim.** FT1's coherent CPM is the user's own protocol design; it is described here as designed and simulated behavior, not an independently measured property.

Architecture footnotes

- **libft1** is Qt-free, built from Fortran + C/C++ over FFTW3, exposing a clean C ABI. It reuses WSJT-X (GPLv3) infrastructure — 77-bit packing, LDPC(174,91), FFTW — and *adds* the FT1 CPM trellis, matched-filter bank, BCJR, Costas sync search, and IR-HARQ combining; DX1 adds the non-coherent M-FSK detector. Rust crates (`ft1-sys` , `ft1`) provide the safe FFI wrapper.
- **Published Windows binaries are cross-compiled beta** (built on Linux/WSL2 targeting Windows via mingw-w64, statically linked so no MinGW runtime DLLs are needed).

10. Summary

FT1 is a coherent 4-CPM turbo modem on a 4-second conversational cycle, sharing the WSJT-X 77-bit payload and LDPC(174,91) FEC with the modes you already know — so the *messages* are familiar even though the *waveform* is not. Its distinguishing bets are **coherence** (more information per second, at the cost of fading fragility), the **4 s cycle** (conversation, not slideshow), and **IR-HARQ** (redundancy that *accumulates* across retransmissions — unique among amateur weak-signal text modes, and, today, designed and simulated rather than live). DX1 backstops the fading fragility with a non-coherent, fading-robust tier on the same payload. FT1 does not out-sensitize FT8; it trades ~3 dB of raw reach for speed and an incremental-redundancy path — and all of these numbers still await the on-air validation that is the project's next, gating step.

Grounding: the waveform/FEC/sync constants come from the modem source — `libft1/include/libft1.h`, `libft1/ft1_cabi.f90`, `libft1/dx1/*.f90`, `crates/ft1-sys/src/lib.rs`, `crates/ft1/src/lib.rs`, and `crates/tempo-core/src/timing.rs`. The IR-HARQ schedule and the simulated throughput/threshold figures in §6 are from the project's Phase-1 IR-HARQ protocol-design analysis (a separate research document), and are explicitly design-and-simulation results, not on-air measurements. See also `docs/ARCHITECTURE.md` and the operator manual (`docs/manual/`).

Author: Seth McCallister, KD9TAW. License: GPL-3.0. Repository: github.com/kd9taw/tempo · Manual: `tempo/docs/manual/` (see `Tiers-FT1-vs-DX1.md`, `Architecture-and-Protocol.md`, `Roadmap.md`).